

problem 1.

- (1)
- For missing data (N/A), we should replace missing values by mean/median of the feature
 - Normalization: we may normalize/rescale features to a range of [0,1]
Standardization: we may re-center features to the mean and scaled by variance.
number of

- (2)
- The curse of dimensionality: when there is a large^V dimensions, the volume of the space increases so fast that data become sparse

Feature selection & dimensionality reduction: PCA.
How: PCA finds a new space with lower dimensions defined by orthogonal principal components.
Why: PCA transform features from the original space to a lower-dimensional space, but principal components record most information with largest eigenvalues.

- (3)
- Stopping condition: specify a desired brightness ($\vec{y}$), and a prediction ($\vec{\hat{y}}$),

$$\text{loss} = \|\vec{\hat{y}} - \vec{y}\|^2$$

stops when the gradient = 0 or descends very slow.

check

iterate

- Computes the gradient of the objective function, find the search direction $p(k)$, with the most negative value, update $x(k+1) = x(k) + \alpha(k)p(k)$
- SGD is named because it only "randomly" samples data at each iteration, and uses a subset of N summand functions.

- (4)
- (2) is optimal
 - (1) is under-fitting with high bias = poor even on training data
 - (3) is overfitting with high variance: good at training data, poor to generalize

- (5)
- Parameters are
- Because the model is trained over training data, and ^V is inclined to training data, maybe overfit training data but poor to generalize.
 - Cross-validation: Split data into training set + validation set + test set, use validation set to further fine-tune training, put better generalization as a goal.

problem 2.

1. (1) function node
- (2) switch node
- (3) ~~project~~ node
- MATT

2. (1) Yes. Because Node-RED is developed by IBM, and the user interface ~~is~~ allows programmers to drag and drop nodes to set up IoT flow.
- (2) No. Node.js is asynchronous.
- (3) No. Import flow can also work.
- (4) Yes. After a flow is created, we could choose some nodes as a subflow.
- (5) No. Only nodes that have messages to be printed can be attached.
- (6) No. Function node can also change/delete/edit messages.
- (7) Yes. Because post and GET are different methods.
- (8) No. Http request node's function is sending Http request to the server, which can be done without a function node.

problem 3.

1. (1) line 3: Bandwidth Roof, memory-bounded
line 1: Computational Roof, compute-bounded

- (2) Bounded by memory: ②
Bounded by computation: ⑤
Bounded by both: ① ⑥
Attainable: ③ ④

- (3) ① ③

- (1) N parameters, each 4B: $4N$ Byte
Latency = $\frac{4N \text{ Byte}}{B \text{ Bytes/s}} = \frac{4N}{B} s$
(index matrix)

- (2) Now: $4M + \frac{\log_2 M}{8} N = 4M + \frac{\log_2 M}{8} N$ Bytes
(FLOAT32 for centroids)

$$N=64, \text{ let } 4M + 8 \log_2 M \leq 256$$

M must be power of 2.

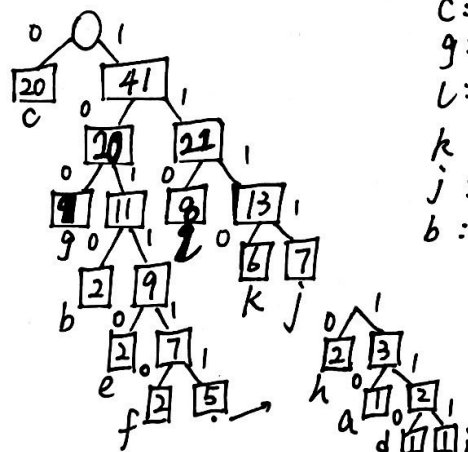
$$M_{\max} = 32$$

- (3) higher. $CTC = \frac{FLOP}{\text{DRAM byte access}}$

FLOP is not changing,
but memory required is reduced

3. (1) pro: Huffman Coding reduce the space needed to store data,
and is lossless data compression
con: Binary ~~strings~~ data are encoded as variable-length code words,
is inconvenient

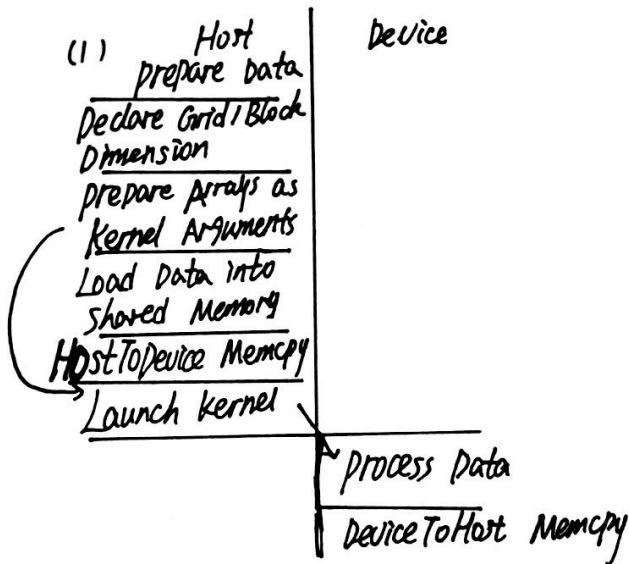
(2)



c: 0
g: 100
l: 110
k: 1110
j: 1111
b: 1010
e: 10110
f: 101110
h: 1011110
a: 10111110
d: 101111110
i: 101111111

- (3) The size of indices to encode is of power of 2, and each index has the same frequency.

problem 4.



- (2)
- opt1: $\frac{640}{16} = 40$ $\frac{960}{24} = 40$ (40, 40) for each thread
- opt2: $\frac{640}{24} = 26.67$ $\frac{960}{24} = 40$ (27, 40) for each thread
- opt3: $\frac{640}{16} = 40$ $\frac{960}{16} = 60$ (40, 60) for each thread

I choose option ~~one~~, because it uses 256 threads (8 warps), each ~~thread~~ is ^{three} responsible for (40, 60) pixels, compared to 1. thread responsible it uses less threads, compared to 2, it will not cause control divergence.

(3) $1000 \times 4 B = 4 KB$

① D

② D

problem 5.

- (1) We need 2 reads ($mat1[i][k], mat2[k][j]$)
1 write ($res[i][j]$) per operation
So we need 12B/op
$$\text{max throughput} = \frac{\text{max bandwidth}}{12B/op}$$

$$\text{max bandwidth} = 3.5 \text{ GFLOPs} \cdot 12B/op = 42 \text{ GB/s}$$

$$\begin{aligned} \text{if } N=256, \# \text{ memory accesses} &= 256 \times 256 \times 256 \times (2+1) \\ &= 50,331,648 \end{aligned}$$

- (2) Mean Latency:
$$\begin{aligned} &((1 - 12.5\%) \times 5 + 12.5\% \times ((1 - 3.13\%) \times 15 + 3.13\% \times ((1 - 100\%) \times 45 + 100\% \times 80))) \\ &= 6.5043125 \text{ ns} \end{aligned}$$

Total Time:
$$6.5043125 \text{ ns} \times 50,331,648 = 0.3275$$

Mean Energy:
$$\begin{aligned} &((1 - 12.5\%) \times 15 + 12.5\% \times ((1 - 3.13\%) \times 26 + 3.13\% \times ((1 - 100\%) \times 47 + 100\% \times 256))) \\ &= 26.289275 \text{ pJ} \end{aligned}$$

Total Energy:
$$26.289275 \text{ pJ} \times 50,331,648 = 1.323 \times 10^3 \text{ J}$$

- (3) Because each L2 cache has 512 KB, it is a ^{small} ~~large~~ amount of memory, so the miss rate is near 100%. ^{relatively}

(4)

1. Challenge: data leaks from IoT systems:
 - Compared with desktop or server systems, because many IoT systems are the integration of end devices, so IoT systems are ~~more~~ more likely to leak data if some end devices are attacked.

2. (a) 1 4
 (b) 5 6 7
 (c) 2
 (d) 3 8

- 3. SGX: provides isolation at the hardware & architecture level
(isolated environments: enclaves; new instructions & hardware)
- Trustzone: provides isolation from the non-secure OS and protects the loaded trusted apps from each other using underlying hardware support
- RZSC-V keystone: Isolation is at the ~~hard~~ architecture level:
 (OP-TEE)
 - RIS-V Privileged ISA
 - PMP

- 4.
- (1) Denial-of-sleep attack will exhaust the battery of the devices or sensors so that IoT system becomes unresponsive. So specifically, it attacks battery.
 - (2) We can just repeatedly send encrypted messages to the device, since encryption/decryption data consume a lot energy, so a large amount of encrypted data will [^]of drain the battery out.

problem 7.

1. (1) $Y = WX + b = 2.5 + 1 + 0.5 = 4$

(2) $E = \frac{1}{2} (Y - Y_{gt})^2 = \frac{1}{2} (4 - 3)^2 = 0.5$

(3) $\frac{dE}{dW} = \frac{dE}{dY} \frac{dY}{dW} = (Y - Y_{gt}) \cdot X = 12.5 \cdot 1$

$\frac{dE}{db} = \frac{dE}{dY} \frac{dY}{db} = (Y - Y_{gt}) \cdot 1 = 1$

2. (1) $Y = W \cdot G(X) + b$

$= 0.5 \times 0.7 + 0.6 + 0.5 = 1.45$

(2) $E = \frac{1}{2} (Y - Y_{gt})^2 = \frac{1}{2} \times (1.45 - 1.5)^2 = 1.25 \times 10^{-3}$

$\frac{dE}{db} = \frac{dE}{dY} \frac{dY}{db}$

$= (Y - Y_{gt}) \cdot 1$

(3) $\frac{dE}{dW} = \frac{dE}{dY} \frac{dY}{dW} = (Y - Y_{gt}) \cdot G(X) = [0, -0.025, -0.05]$

$= -0.05$

(4) $\frac{dE}{dX} = \frac{dE}{dY} \frac{dY}{dX} = (Y - Y_{gt}) \cdot W = [-0.04, -0.035, -0.03]$

3. Yes. Use unrolled matrix multiplication:

- Calculate the horizontal matrix index: $w_unroll = h \times w_out + w$;

- find the beginning of the unrolled: $w_base = C \times (k \times k)$;

- Calculate the vertical matrix index: $h_unroll = w_base + p \times k + q$;

$X_unroll[b, h_unroll, w_unroll] = X[b, c, h+p, w+q]$

problem 8.

$$1. (1) F(2,3) = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix}$$

first row and second should be multiplied by $[90, 91, 92]^T$

(2) In hardware design, a multiplier ~~is~~ ^{requires} more space and energy than an adder (for example, some hardware multiplier designs have shift-registers, adders, and etc.), so when hardware resources are limited, we should save resources by replacing "expensive" multiplications with "cheap" additions.

$$(3) \cdot \# \text{ of multiplications} = \cancel{4 \times 4} = 16$$

$$= 5$$

Because # of multiplications of Winograd =

$$\# \text{ of rows of } G = \# \text{ of rows of } B^T$$

(indices of g_0, g_1, g_2) (indices of $d_0, d_1, \dots, d_4$)

$$\cdot \# \text{ of multiplications (standard conv)} = \cancel{3 \times 3 \times 3} = 27$$

so Winograd saves: $36 - 16 = 20$ multiplications

$$\cancel{9} =$$

$$9 - 5 = 4$$

problem 7.

1. Amdahl's Law = $Speedup = \frac{T_{new}}{T_{old}} = \frac{1}{(1 - \text{Fraction parallel}) + \frac{\text{Fraction parallel}}{Speedup_{parallel}}}$

$\text{Fraction parallel} = 1$

$Speedup_{parallel} = 1500 \times 1500$ (each thread for 1 multiplication)

$Speedup = \frac{1}{(1-1) + \frac{1}{1500 \times 1500}} = 1500^2 = 2,250,000$

2. kernel 1 : Basic. Since it uses large amount of main memory but small amount of shared memory.

kernel 2 : Tiled. Since it uses more L1/L2 caches and less main memory, which means it uses shared memory for tiled matrix multiplication.

3.

• kernel performance

$= \frac{1500 \times 1500 \times 1500 \times 2 \text{ FLOP}}{3.394 \text{ ms}}$

$= 1.989 \text{ e}^9 \text{ FLOPs} = 1.989 \text{ TFLOPs}$

• for example, under load unbalanced situations, overlapping memory transfer may not be beneficial since the total latency depends on the longest work to do.

load balanced / unbalanced

